

Q517-2

# GREEDY ALGO

|                  | m <sub>1</sub>              | m <sub>2</sub> | m <sub>3</sub> | m <sub>4</sub> | m <sub>5</sub> | m <sub>6</sub> |
|------------------|-----------------------------|----------------|----------------|----------------|----------------|----------------|
| start            | 7                           | 5              | 8              | 2              | 5              | 3              |
| end              | 9                           | 10             | 12             | 8              | 7              | 6              |
| sort acc. ending | max. no. meeting <u>can</u> |                |                | attend?        |                |                |

|       | m <sub>6</sub> | m <sub>5</sub> | m <sub>4</sub> | m <sub>1</sub> | m <sub>2</sub> | m <sub>3</sub> |
|-------|----------------|----------------|----------------|----------------|----------------|----------------|
| start | 3              | 5              | 2              | 7              | 5              | 8              |
| end   | 6              | 7              | 8              | 9              | 10             | 12             |

2

V.I.

# N meeting in one room

|        |     |    |     |
|--------|-----|----|-----|
| start. | [0  | 20 | 12] |
| end    | [20 | 30 | 25] |

max.

↓  
[[10, 20], [20, 30], [12, 25]]

[ (0, 10, 20) ]  
meeting Id    starting    ending

[ (0, 10, 20) ]

meeting = []

for i in range(len(start))

meeting.append((i, start[i], end[i]))

[(0, 10, 20), (1, 20, 30), (2, 12, 25)]

sort?  
↳ Ending

acc. which part want sorting

Learn this ✓ sorted\_meeting = sorted(meeting, key=lambda x: x[2])

inc. order

starting

[(0, 10, 20), (2, 12, 25), (1, 20, 30)]

ending times

for

⇒ last-meeting = 20  
count = 1

for (id, start, end) in sorted-meeting:

if (start >= last-meeting):

count = count + 1

last-meeting = end

ex:

day sun

meeting ID

end

start  
end

5  
9

1  
2

8  
9

3  
4

0  
6

5  
7

[(0, 5, 9), (1, 1, 2), (2, 8, 9), (3, 3, 4), (4, 0, 6), (5, 5, 7)]

[ (meeting id, start, end) ]

0 1 2

[(1, 1, 2), (3, 3, 4), (4, 0, 6), (5, 5, 7), (0, 5, 9), (2, 8, 9)]

⇒ m = [1]

⇒ last-meeting-end = sorted-meeting[0][2] = 2

count = 1

for (id, start, end) in sorted-meeting:



if ( start  $\geq$  last-meeting-end )

count = count + 1

last-meeting = end  
m.append(id)

(1, 1, 2)

if start  $\geq$  last meeting-end

1  $\geq$  2 (False)

(3, 3, 4)

start  $\geq$  last-m-e

3  $\geq$  2 (True)

$\rightarrow$  count = count + 1 = 2

last-end = 4

(4, 0, 6)

start  $\geq$  last-m-e

0  $\geq$  4 (F)

✓  
(5, 5, 7)

start  $\geq$  last-m-e

5  $\geq$  4 (T)

$\hookrightarrow \text{count} = \text{count} + 1 = 2 + 1 = 3$   
 $\text{last\_m\_e} = 7$

$(0, 5, 9)$   
 $\uparrow$

$\text{start} \geq \text{last\_m\_e}$   
 $5 \geq 7 \quad (F)$

$\checkmark$   
 $\downarrow$   
 $(2, 8, 9)$   
 $\uparrow$

$\text{start} \geq \text{last\_m\_e}$   
 $8 \geq 7 \quad (T)$

$\hookrightarrow \text{count} = \text{count} + 1 = 3 + 1 = 4$   
 $\text{last\_m\_e} = 9$

$(0, 5, 9)$   $(2, 8, 9)$   
 $(2, 5, 9)$   $(0, 8, 9)$   
 $\downarrow$

~~4~~  
~~return~~  $m\_id$   $\checkmark$

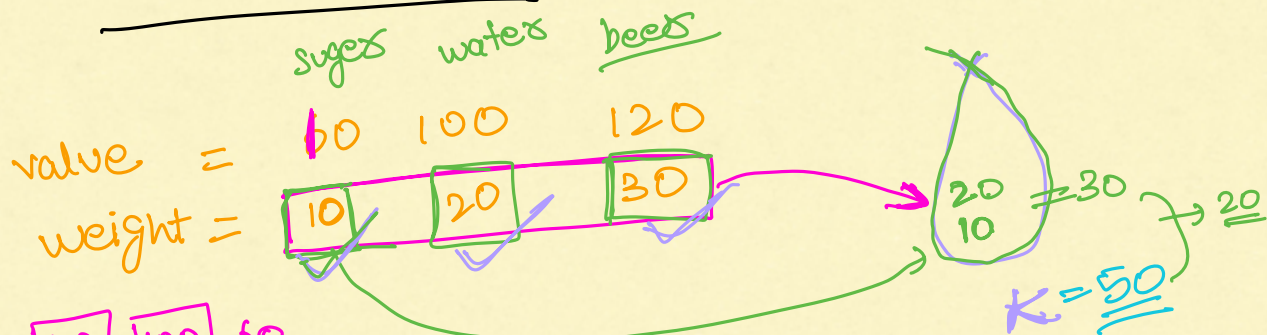
$[1, 3, 4, 7]$

$\uparrow$   
 U can't break the item

$\hookrightarrow$  0-1 knapsack CDP

\*\*\*

# # Fractional Knapsack



|     |     |    |
|-----|-----|----|
| 120 | 100 | 60 |
| 30  | 20  | 10 |

$\frac{220}{20} = 11$   
~~X~~

60  
 100  
 $20 \times 4 = 80$   
240

$\frac{60}{10} = 6 \rightarrow \text{unit}$   
 $5 \times 6 = 30$



ex:  $\frac{99}{100} = 99\%$   
 $\frac{9}{10} = 90\%$

60  
10

100  
20

120  
30

rem  
10

$\frac{60}{10} = 6$

$\frac{100}{20} = 5$

$\frac{120}{30} = 4$

ratio

order

$[6, 5, 4]$

$6 \times 10$

$60 + 5 \times 20 = 100 + 20 \times 4$

$60 + 100 + 80 = 240$

$\frac{50}{20} = 2.5$

20



ex: value = [60, 100]  
 w = [10, 20] → 10

20 K=50

P=?  
 160 →

ratio = [60/10=6, 100/20=5]

↓ order = [6, 5]

60+100 = 160

100/20 = 5  
 1 momo  
 5 × 10 = 50

10 × 100 = 1000

id/idx ratio  
 [(0, 6), (1, 5)]

value/weight \* 8

if weight[0] <= bag  
 profit = w \* value  
 bag = bag - weight[0]

else  
~~weight[0]~~  
 120 \* 8 = 4 \* 10 =

120  
 30

10  
 10

8=10

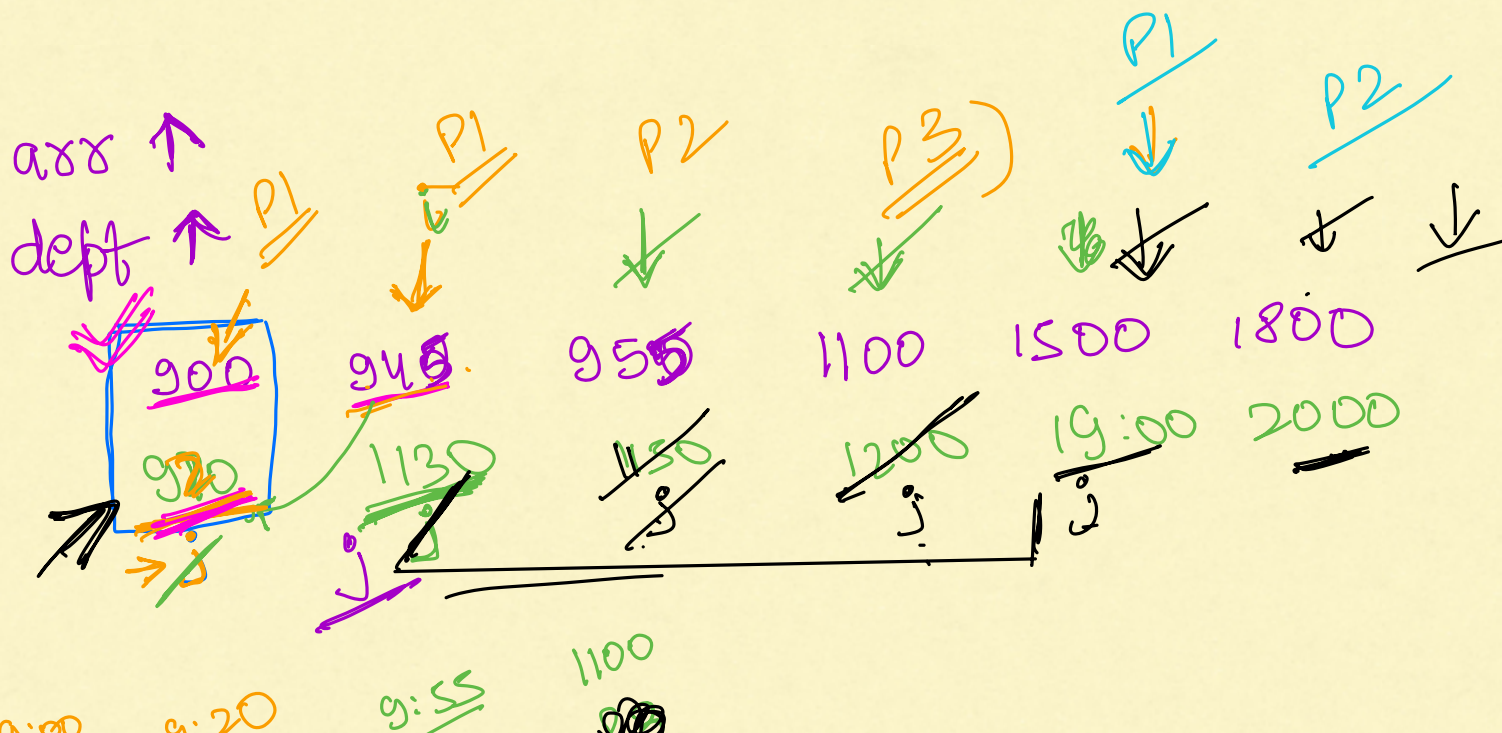
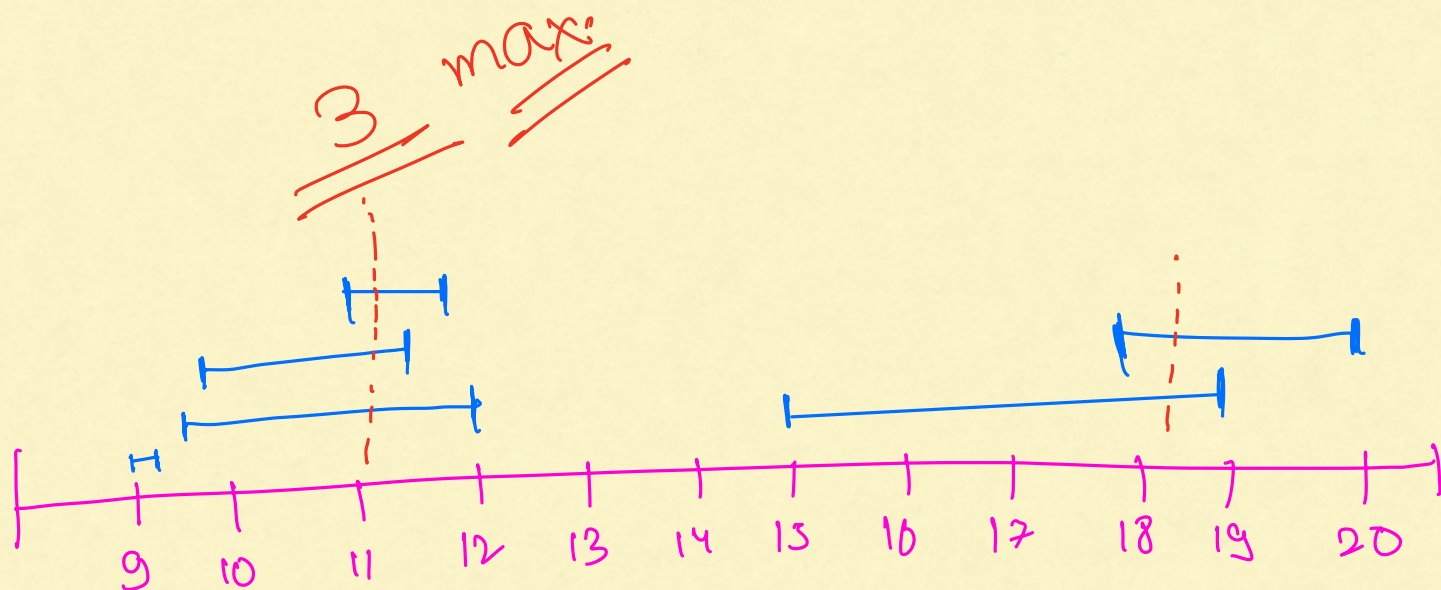
Platform → Room

<sup>no. of</sup> ~~OPPE~~ # Min. no. of plat form required for a railway.

arr = [9:00, 9:45, 9:55, 11:00, 15:00, 18:00]

dep = [9:20, 12:00, 11:30, 11:50, 19:00, 20:00]

Find the min. no. of platforms required





P1

P1

P2

max = 3

arr.sort()

dep.sort()

~~PF = 1~~

PF = 1

count = 1

|                | a    | d         |
|----------------|------|-----------|
| 900            | 945  | 955 1100  |
| 920            | 1130 | 1150 1200 |
| <del>940</del> | d    |           |

count = 3

a = 1

d = 0

while (a < len(arr) and d < len(dep))

if arr[a] <= dep[d]:

count = count + 1

a = a + 1

else:

count = count - 1

d = d + 1

if (count > PF)  
PF = count

return @PF

| <u>Time</u> | <u>Event</u> | <u>Total PF count</u> |      |
|-------------|--------------|-----------------------|------|
| 9:00        | Axx          | <del>1</del>          | PF=1 |
| 9:10        | Dept         | 0                     | PF=1 |
| 9:40        | Axx          | 1                     | PF=1 |
| 9:50        | Axx          | 2                     | PF=2 |
| 11:00       | Axx          | 3                     | PF=3 |
| 11:20       | Dept         | 2                     | PF=3 |
| 11:30       | Dept         | 1                     | PF=3 |
| 12:00       | Dept         | 0                     | PF=3 |
| 15:00       | Axx          | 1                     | PF=3 |
| 18:00       | Axx          | 2                     | PF=3 |
| 19:00       | Dept         | 1                     | PF=3 |
| 20:00       | Dept         | 0                     | PF=3 |

return PF